
Ultime Team Manager

Login paso a paso

Formulario, validacion, estado y navegacion protegida en Flutter

Stack: Flutter, Dart, Riverpod, go_router, shared_preferences.

Objetivo: construir un login solido, offline y escalable a online, en **6 fases** incrementales. En cada fase la app sigue compilando.

Integrantes: Sebastian Ramirez • Joaquin Parraud.

Índice general

Introduccion	2
1. Dependencias y arranque	3
1.1. Anadir las dependencias	3
1.2. Activar Riverpod en el arranque	3
2. El dominio: entidad y contrato	5
2.1. La entidad AppUser	5
2.2. El contrato AuthRepository	5
3. Repositorio local con sesion persistente	7
3.1. AuthRepositoryLocal	7
4. Estado con Riverpod	9
4.1. Providers y estado de autenticacion	9
5. La pantalla de login	11
5.1. LoginScreen	11
5.2. HomeScreen (destino tras el login)	13
5.3. Ver el login en marcha (temporal)	14
6. Navegacion protegida con go_router	15
6.1. El router con guardia de sesion	15
6.2. Conectar el router en main.dart	16
Resumen de archivos	17

Introduccion

Vamos a construir la pantalla de login de **Ultime Team Manager**. No es un formulario suelto: es un login *de verdad*, con arquitectura por capas, estado gestionado con Riverpod, sesion persistente y navegacion protegida. Hoy funciona **offline**; manana, cambiando una sola clase, funcionara contra Supabase.

El flujo que vamos a montar

1. La app arranca y **restaura** la sesion guardada (si la hay).
2. Si no hay sesion, el `go_router` nos lleva a `LoginScreen`.
3. El usuario escribe correo y contrasena. El formulario **valida** los campos.
4. Al pulsar *Entrar*, el `AuthController` llama al repositorio, muestra un **spinner** y guarda la sesion si las credenciales son correctas.
5. El router detecta el cambio de estado y **redirige** a `HomeScreen`.
6. Desde Home, *Cerrar sesion* borra los datos y vuelve al login.

Las 6 fases

1. **Dependencias y arranque** — anadimos paquetes y activamos Riverpod.
2. **El dominio** — la entidad `AppUser` y el contrato `AuthRepository`.
3. **Repositorio local** — login offline con sesion persistente.
4. **Estado con Riverpod** — el `AuthController` y sus providers.
5. **La pantalla de login** — formulario, validacion y estados de carga/error.
6. **Navegacion protegida** — `go_router` con guardia de sesion.

Pruebalo

Credenciales de demo (modo offline): correo `demo@ultime.com`, contrasena `123456`. Las definiremos en la Fase 3 y podras cambiarlas.

Que hace este bloque

Nota sobre el nombre del paquete. El proyecto base se llama `contador_app`, por eso todos los `import` usan `package:contador_app/...`. Si mas adelante renombras el paquete, actualiza esos imports de golpe.

Fase 1

Dependencias y arranque

Antes de escribir logica, dejamos el proyecto listo: anadimos las dependencias y envolvemos la app en un `ProviderScope` (el contenedor global de estado de Riverpod).

1.1 Anadir las dependencias

Donde insertarlo

Archivo `pubspec.yaml`. Dentro de la seccion `dependencies:`, justo **debajo** de la linea `cupertino_icons: ^1.0.8`.

```
1 dependencies:
2   flutter:
3     sdk: flutter
4   cupertino_icons: ^1.0.8
5   flutter_riverpod: ^2.6.1      # gestion de estado
6   go_router: ^14.6.2           # navegacion declarativa
7   shared_preferences: ^2.3.3    # persistencia de la sesion
```

Que hace este bloque

- `flutter_riverpod`: providers y *notifiers* para el estado de autenticacion.
- `go_router`: rutas por URL y *redirect* (la guardia de sesion).
- `shared_preferences`: guarda la sesion en el dispositivo (clave-valor).

Ahora descarga los paquetes desde la terminal:

```
flutter pub get
```

1.2 Activar Riverpod en el arranque

Donde insertarlo

Archivo `lib/main.dart`. Sustituye el `import` y la funcion `main()` actuales por lo siguiente (dejamos el resto del archivo igual por ahora).

```
1 import 'package:flutter/material.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3 import 'package:contador_app/presentation/screens/counter/counter_screen.dart';
4
5 // Envolvemos la app en ProviderScope: sin el, ningun provider funciona.
6 void main() => runApp(const ProviderScope(child: MyApp()));
```

Que hace este bloque

`ProviderScope` es la raiz donde Riverpod guarda el estado de todos los providers. Debe envolver a *toda* la app. De momento `MyApp` sigue mostrando el contador; lo cambiaremos en la Fase 5.

Pruebalo

Ejecuta `flutter run`. La app debe seguir mostrando el contador sin errores. Si compila, la base de Riverpod ya esta puesta.

Fase 2

El dominio: entidad y contrato

La capa de dominio no sabe nada de Flutter ni de como se guardan los datos. Define **que** es un usuario y **que operaciones** de autenticacion existen. Asi, cambiar de “offline” a “Supabase” no toca ni la UI ni el dominio.

2.1 La entidad AppUser

Donde insertarlo

Crea el archivo `lib/domain/entities/app_user.dart` con este contenido.

```
1 // Representa al usuario autenticado dentro de la app.
2 class AppUser {
3     final String id;
4     final String email;
5
6     const AppUser({required this.id, required this.email});
7 }
```

Que hace este bloque

Un objeto inmutable (`final` + constructor `const`) con lo minimo que necesitamos del usuario. Nada de contraseñas aqui: la entidad viaja por la UI.

2.2 El contrato AuthRepository

Donde insertarlo

Crea el archivo `lib/domain/repositories/auth_repository.dart`.

```
1 import 'package:contador_app/domain/entities/app_user.dart';
2
3 // Contrato de autenticacion. La UI depende de esta abstraccion,
4 // no de una implementacion concreta (hoy local, manana Supabase).
5 abstract class AuthRepository {
6     // Devuelve el usuario si hay sesion guardada, o null.
7     Future<AppUser?> currentUser();
8
9     // Inicia sesion. Lanza AuthException si las credenciales fallan.
10    Future<AppUser> signIn({required String email, required String password});
11
12    // Cierra la sesion y borra los datos guardados.
13    Future<void> signOut();
14 }
15
16 // Error de dominio para credenciales incorrectas u otros fallos de auth.
17 class AuthException implements Exception {
18     final String message;
19     const AuthException(this.message);
20 }
```

```
20
21     @Override
22     String toString() => message;
23 }
```

Que hace este bloque

- `AuthRepository` es **abstracto**: solo declara *que* se puede hacer, no *como*. En la Fase 3 escribimos la version offline.
- `currentUser()` permite restaurar la sesion al abrir la app.
- `AuthException` nos deja distinguir un error de credenciales de un *bug*, y mostrar un mensaje claro al usuario.

Fase 3

Repositorio local con sesion persistente

Aqui implementamos el contrato para el modo **offline**: validamos contra un usuario de demo y guardamos la sesion con `shared_preferences`.

3.1 AuthRepositoryLocal

Donde insertarlo

Crea el archivo `lib/data/repositories/auth_repository_local.dart`.

```
1  import 'package:shared_preferences/shared_preferences.dart';
2  import 'package:contador_app/domain/entities/app_user.dart';
3  import 'package:contador_app/domain/repositories/auth_repository.dart';
4
5  // Implementacion OFFLINE del contrato de autenticacion.
6  class AuthRepositoryLocal implements AuthRepository {
7    // Claves con las que guardamos la sesion en el dispositivo.
8    static const _kUserId = 'auth_user_id';
9    static const _kUserEmail = 'auth_user_email';
10
11    // Usuario de demo para el modo offline. Cambialo si quieres.
12    static const _demoEmail = 'demo@ultime.com';
13    static const _demoPassword = '123456';
14
15    @override
16    Future<AppUser?> currentUser() async {
17      final prefs = await SharedPreferences.getInstance();
18      final id = prefs.getString(_kUserId);
19      final email = prefs.getString(_kUserEmail);
20      if (id == null || email == null) return null;
21      return AppUser(id: id, email: email);
22    }
23
24    @override
25    Future<AppUser> signIn({
26      required String email,
27      required String password,
28    }) async {
29      // Simula la latencia de una peticion de red real.
30      await Future.delayed(const Duration(milliseconds: 600));
31
32      final normalized = email.trim().toLowerCase();
33      if (normalized != _demoEmail || password != _demoPassword) {
34        throw const AuthException('Correo o contrasena incorrectos');
35      }
36
37      final user = AppUser(id: 'u1', email: normalized);
38      final prefs = await SharedPreferences.getInstance();
39      await prefs.setString(_kUserId, user.id);
40      await prefs.setString(_kUserEmail, user.email);
41      return user;
```



```
42     }
43
44     @override
45     Future<void> signOut() async {
46         final prefs = await SharedPreferences.getInstance();
47         await prefs.remove(_kUserId);
48         await prefs.remove(_kUserEmail);
49     }
50 }
```

Que hace este bloque

- **signIn()** normaliza el correo, simula un retardo (para que el spinner se vea), y compara con las credenciales de demo. Si fallan, lanza **AuthException**; si aciertan, **guarda** la sesion.
- **currentUser()** reconstruye el **AppUser** desde lo guardado.
- **signOut()** borra las claves: sesion cerrada.

Pruebalo

Escalable a online: cuando quieras Supabase, creas **AuthRepositorySupabase** implements **AuthRepository** y cambias *una linea* en el provider de la Fase 4. Ni la UI ni el dominio se enteran.

Fase 4

Estado con Riverpod

El `AuthController` es el cerebro del login: guarda el estado (cargando, error, usuario) y expone las acciones `signIn` y `signOut`. La UI solo *observa* este estado.

4.1 Providers y estado de autenticacion

Donde insertarlo

Crea el archivo `lib/presentation/providers/auth_provider.dart`.

```
1 import 'package:flutter_riverpod/flutter_riverpod.dart';
2 import 'package:contador_app/domain/entities/app_user.dart';
3 import 'package:contador_app/domain/repositories/auth_repository.dart';
4 import 'package:contador_app/data/repositories/auth_repository_local.dart';
5
6 // 1) Provider del repositorio. Para pasar a online, cambia SOLO esta linea
7 // por AuthRepositorySupabase().
8 final authRepositoryProvider = Provider<AuthRepository>((ref) {
9   return AuthRepositoryLocal();
10 });
11
12 // 2) Estados posibles de la sesion.
13 enum AuthStatus { unknown, authenticated, unauthenticated }
14
15 // 3) El estado inmutable que observa la UI.
16 class AuthState {
17   final AuthStatus status;
18   final AppUser? user;
19   final bool isSubmitting; // true mientras se procesa el login
20   final String? errorMessage;
21
22   const AuthState({
23     this.status = AuthStatus.unknown,
24     this.user,
25     this.isSubmitting = false,
26     this.errorMessage,
27   });
28
29   AuthState copyWith({
30     AuthStatus? status,
31     AppUser? user,
32     bool? isSubmitting,
33     String? errorMessage,
34   }) {
35     return AuthState(
36       status: status ?? this.status,
37       user: user ?? this.user,
38       isSubmitting: isSubmitting ?? this.isSubmitting,
39       errorMessage: errorMessage, // se limpia si no se pasa
40     );
41   }
42 }
43
```

```

44 // 4) El controlador: mantiene el AuthState y expone las acciones.
45 final authControllerProvider =
46     NotifierProvider<AuthController, AuthState>(AuthController.new);
47
48 class AuthController extends Notifier<AuthState> {
49     @override
50     AuthState build() {
51         _restoreSession();           // restaura la sesion al arrancar
52         return const AuthState();    // estado inicial: unknown
53     }
54
55     AuthRepository get _repo => ref.read(authRepositoryProvider);
56
57     Future<void> _restoreSession() async {
58         final user = await _repo.currentUser();
59         state = state.copyWith(
60             status: user != null
61                 ? AuthStatus.authenticated
62                 : AuthStatus.unauthenticated,
63             user: user,
64         );
65     }
66
67     // Devuelve true si el login fue correcto.
68     Future<bool> signIn(String email, String password) async {
69         state = state.copyWith(isSubmitting: true); // enciende el spinner
70         try {
71             final user = await _repo.signIn(email: email, password: password);
72             state = state.copyWith(
73                 status: AuthStatus.authenticated,
74                 user: user,
75                 isSubmitting: false,
76             );
77             return true;
78         } on AuthException catch (e) {
79             state = state.copyWith(isSubmitting: false, errorMessage: e.message);
80             return false;
81         }
82     }
83
84     Future<void> signOut() async {
85         await _repo.signOut();
86         state = const AuthState(status: AuthStatus.unauthenticated);
87     }
88 }

```

Que hace este bloque

- (1) authRepositoryProvider es el *unico* punto que conoce la implementacion concreta. Cambiarlo = cambiar de backend.
- (3) AuthState agrupa todo lo que la UI necesita pintar: estado, usuario, si esta enviando y el mensaje de error. Es inmutable; copyWith crea copias con cambios puntuales.
- (4) AuthController enciende isSubmitting antes de llamar al repositorio, y actualiza state segun el resultado. build() lanza la restauracion de sesion al crearse.
- errorMessage se limpia en cada copyWith que no lo pase: es un error *transitorio*, no queremos que se quede pegado.

Fase 5

La pantalla de login

Toca la UI: un formulario con validacion, un boton que muestra spinner mientras carga y un mensaje de error si las credenciales fallan. Tambien creamos una Home sencilla como destino tras entrar.

5.1 LoginScreen

Donde insertarlo

Crea el archivo `lib/presentation/screens/auth/login_screen.dart`.

```
1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3  import 'package:contador_app/presentation/providers/auth_provider.dart';
4
5  // Usamos ConsumerStatefulWidget: necesitamos estado local (los controllers
6  // de texto) y ademas leer/observar providers de Riverpod.
7  class LoginScreen extends ConsumerStatefulWidget {
8    const LoginScreen({super.key});
9
10   @override
11   ConsumerState<LoginScreen> createState() => _LoginScreenState();
12 }
13
14 class _LoginScreenState extends ConsumerState<LoginScreen> {
15   final _formKey = GlobalKey<FormState>();
16   final _emailCtrl = TextEditingController();
17   final _passCtrl = TextEditingController();
18   bool _obscure = true;
19
20   @override
21   void dispose() {
22     _emailCtrl.dispose();
23     _passCtrl.dispose();
24     super.dispose();
25   }
26
27   String? _validateEmail(String? value) {
28     final text = value?.trim() ?? '';
29     if (text.isEmpty) return 'Introduce tu correo';
30     final regex = RegExp(r'^[\w.\-]+\.[\w.\-]+\.[\w.\-]+\.$');
31     if (!regex.hasMatch(text)) return 'Correo no valido';
32     return null;
33   }
34
35   String? _validatePassword(String? value) {
36     final text = value ?? '';
37     if (text.isEmpty) return 'Introduce tu contrasena';
38     if (text.length < 6) return 'Minimo 6 caracteres';
39     return null;
40   }
41
42   Future<void> _submit() async {
```

```

43 FocusScope.of(context).unfocus(); // oculta el teclado
44 if (!_formKey.currentState!.validate()) return;
45
46 final ok = await ref
47   .read(authControllerProvider.notifier)
48   .signIn(_emailCtrl.text, _passCtrl.text);
49
50 if (!ok && mounted) {
51   final msg = ref.read(authControllerProvider).errorMessage ?? 'Error';
52   ScaffoldMessenger.of(context)
53     .showSnackBar(SnackBar(content: Text(msg)));
54 }
55 // Si ok == true, el router (Fase 6) redirige solo a Home.
56 }
57
58 @override
59 Widget build(BuildContext context) {
60   // Observamos solo isSubmitting para redibujar el boton/campos.
61   final isSubmitting =
62     ref.watch(authControllerProvider.select((s) => s.isSubmitting));
63
64   return Scaffold(
65     body: SafeArea(
66       child: Center(
67         child: SingleChildScrollView(
68           padding: const EdgeInsets.all(24),
69           child: Form(
70             key: _formKey,
71             child: Column(
72               mainAxisAlignment: MainAxisAlignment.min,
73               crossAxisAlignment: CrossAxisAlignment.stretch,
74               children: [
75                 const Icon(Icons.sports_soccer, size: 72, color: Colors.green),
76                 const SizedBox(height: 16),
77                 Text(
78                   'Ultime Team Manager',
79                   textAlign: TextAlign.center,
80                   style: Theme.of(context).textTheme.headlineSmall,
81                 ),
82                 const SizedBox(height: 32),
83                 TextFormField(
84                   controller: _emailCtrl,
85                   keyboardType: TextInputType.emailAddress,
86                   textInputAction: TextInputAction.next,
87                   enabled: !isSubmitting,
88                   decoration: const InputDecoration(
89                     labelText: 'Correo',
90                     prefixIcon: Icon(Icons.email_outlined),
91                     border: OutlineInputBorder(),
92                   ),
93                   validator: _validateEmail,
94                 ),
95                 const SizedBox(height: 16),
96                 TextFormField(
97                   controller: _passCtrl,
98                   obscureText: _obscure,
99                   enabled: !isSubmitting,
100                   textInputAction: TextInputAction.done,
101                   onFieldSubmitted: (_) => _submit(),
102                   decoration: InputDecoration(
103                     labelText: 'Contraseña',
104                     prefixIcon: const Icon(Icons.lock_outline),
105                     border: const OutlineInputBorder(),

```

```

106         suffixIcon: IconButton(
107             icon: Icon(_obscure
108                 ? Icons.visibility_off
109                 : Icons.visibility),
110             onPressed: () => setState(() => _obscure = !_obscure),
111         ),
112     ),
113     validator: _validatePassword,
114 ),
115     const SizedBox(height: 24),
116     FilledButton(
117         onPressed: isSubmitting ? null : _submit,
118         child: isSubmitting
119             ? const SizedBox(
120                 height: 20,
121                 width: 20,
122                 child: CircularProgressIndicator(strokeWidth: 2),
123             )
124             : const Text('Entrar'),
125     ),
126     const SizedBox(height: 12),
127     Text(
128         'Demo: demo@ultime.com / 123456',
129         textAlign: TextAlign.center,
130         style: Theme.of(context).textTheme.bodySmall,
131     ),
132 ],
133 ),
134 ),
135 ),
136 ),
137 ),
138 );
139 }
140 }

```

Que hace este bloque

- Form + _formKey: validate() ejecuta todos los validator y pinta los errores bajo cada campo.
- _validateEmail/_validatePassword: reglas de validacion (formato de correo y minimo 6 caracteres).
- isSubmitting deshabilita los campos y convierte el boton en un spinner: **estado de carga**.
- En error mostramos un **SnackBar**; dispose() libera los controllers para no tener fugas de memoria.

5.2 HomeScreen (destino tras el login)

Donde insertarlo

Crea el archivo `lib/presentation/screens/home/home_screen.dart`.

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3  import 'package:contador_app/presentation/providers/auth_provider.dart';
4
5  class HomeScreen extends ConsumerWidget {

```

```

6      const HomeScreen({super.key});
7
8      @override
9      Widget build(BuildContext context, WidgetRef ref) {
10         final user = ref.watch(authControllerProvider).user;
11
12         return Scaffold(
13           appBar: AppBar(
14             title: const Text('Inicio'),
15             actions: [
16               IconButton(
17                 icon: const Icon(Icons.logout),
18                 onPressed: () =>
19                   ref.read(authControllerProvider.notifier).signOut(),
20               ),
21             ],
22           ),
23           body: Center(
24             child: Text('Bienvenido, ${user?.email ?? 'jugador'!}'),
25           ),
26         );
27       }
28     }

```

Que hace este bloque

ConsumerWidget porque solo necesita *leer* providers. El boton de la AppBar llama a signOut(): al cambiar el estado, el router nos devolviera al login.

5.3 Ver el login en marcha (temporal)

Donde insertarlo

Archivo lib/main.dart. Cambia el import del contador por el del login, y pon LoginScreen como home. Es temporal: en la Fase 6 lo sustituye el router.

```

1  // Sustituye esta linea:
2  import 'package:contador_app/presentation/screens/counter/counter_screen.dart';
3  // por esta:
4  import 'package:contador_app/presentation/screens/auth/login_screen.dart';
5
6  // ...y dentro de MaterialApp cambia:
7  //   home: const CounterScreen(),
8  // por:
9  home: const LoginScreen(),

```

Pruebalo

Ejecuta flutter run. Prueba a dejar campos vacios o un correo mal escrito: deben salir los errores de validacion. Con credenciales incorrectas, veras el **SnackBar** rojo. Con las de demo el login *funciona*, pero aun no navega a Home: eso lo arregla la Fase 6.

Fase 6

Navegacion protegida con go_router

Ultima pieza: el router decide a que pantalla ir segun el estado de la sesion. Si no hay sesion, al login; si la hay, a Home. Y reacciona **automaticamente** cuando el estado cambia.

6.1 El router con guardia de sesion

Donde insertarlo

Crea el archivo `lib/config/router/app_router.dart`.

```
1 import 'package:flutter/foundation.dart';
2 import 'package:flutter_riverpod/flutter_riverpod.dart';
3 import 'package:go_router/go_router.dart';
4 import 'package:contador_app/presentation/providers/auth_provider.dart';
5 import 'package:contador_app/presentation/screens/auth/login_screen.dart';
6 import 'package:contador_app/presentation/screens/home/home_screen.dart';
7
8 final routerProvider = Provider<GoRouter>((ref) {
9   // Un notifier que "despierta" al router cuando cambia el estado de sesion.
10   final refresh = ValueNotifier<AuthStatus>(AuthStatus.unknown);
11   ref.listen(
12     authControllerProvider.select((s) => s.status),
13     (previous, next) => refresh.value = next,
14     fireImmediately: true,
15   );
16   ref.onDispose(refresh.dispose);
17
18   return GoRouter(
19     initialLocation: '/login',
20     refreshListenable: refresh,
21     redirect: (context, state) {
22       final status = ref.read(authControllerProvider).status;
23
24       // Mientras restauramos la sesion, no decidimos nada todavia.
25       if (status == AuthStatus.unknown) return null;
26
27       final loggingIn = state.matchedLocation == '/login';
28       final loggedIn = status == AuthStatus.authenticated;
29
30       if (!loggedIn) return loggingIn ? null : '/login';
31       if (loggingIn) return '/home';
32       return null;
33     },
34     routes: [
35       GoRoute(path: '/login', builder: (_, __) => const LoginScreen()),
36       GoRoute(path: '/home', builder: (_, __) => const HomeScreen()),
37     ],
38   );
39 });
```


Que hace este bloque

- **redirect** es la **guardia**: si no hay sesion te manda a /login; si ya entraste, te saca del login a /home.
- Con `status == unknown` devolvemos null: esperamos a que termine la restauracion antes de redirigir (evita parpadeos).
- `refreshListenable + ref.listen`: cuando el `AuthController` cambia el estado, el `ValueNotifier` avisa al router y este vuelve a evaluar el `redirect`. Asi el login y el logout navegan solos.

6.2 Conectar el router en main.dart

Donde insertarlo

Archivo `lib/main.dart`. Reemplaza **todo** el contenido por esta version final.

```

1  import 'package:flutter/material.dart';
2  import 'package:flutter_riverpod/flutter_riverpod.dart';
3  import 'package:contador_app/config/router/app_router.dart';
4
5  void main() => runApp(const ProviderScope(child: MyApp()));
6
7  // Ahora es ConsumerWidget para poder leer el routerProvider.
8  class MyApp extends ConsumerWidget {
9    const MyApp({super.key});
10
11    @override
12    Widget build(BuildContext context, WidgetRef ref) {
13      final router = ref.watch(routerProvider);
14      return MaterialApp.router(
15        debugShowCheckedModeBanner: false,
16        theme: ThemeData(useMaterial3: true, colorSchemeSeed: Colors.green),
17        routerConfig: router,
18      );
19    }
20  }

```

Que hace este bloque

Pasamos de `MaterialApp` con `home`: a `MaterialApp.router` con `routerConfig: .MyApp` ahora es `ConsumerWidget` para poder `watch` del `routerProvider`. Ya no importamos ninguna pantalla directamente: manda el router.

Pruebalo

Prueba completa:

1. Arranca: te lleva al login (no hay sesion).
 2. Entra con `demo@ultime.com / 123456`: navega a Home solo.
 3. **Cierra la app y vuelve a abrirla**: entra directo a Home (sesion persistente).
 4. Pulsa *Cerrar sesion*: vuelves al login.
- Si todo esto ocurre, tienes un login completo, persistente y escalable a online.

Resumen de archivos

- `pubspec.yaml` — 3 dependencias nuevas (*modificado*).
- `lib/main.dart` — `ProviderScope` y `MaterialApp.router` (*modificado*).
- `lib/domain/entities/app_user.dart` — entidad `AppUser`.
- `lib/domain/repositories/auth_repository.dart` — contrato + `AuthException`.
- `lib/data/repositories/auth_repository_local.dart` — login offline.
- `lib/presentation/providers/auth_provider.dart` — providers y `AuthController`.
- `lib/presentation/screens/auth/login_screen.dart` — formulario y validacion.
- `lib/presentation/screens/home/home_screen.dart` — destino tras el login.
- `lib/config/router/app_router.dart` — navegacion protegida.

Ultime Team Manager — Login paso a paso — Sebastian Ramirez y Joaquin Parraud.